

INTEGRACION DE SISTEMAS

Tema: Taller Transferencia de Archivos 1

Integrantes:

Paulo Olivo

Romel Gualoto

Isaac Trujillo

Brandon Arellano

Ingeniería de Software

Fecha: 07/10/2026

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

ISWZ 3401– INTELIGENCIA ARTIFICIAL I - [5448] - 202620

Taller Transferencia de archivos 1

Autores (Gualoto Romel, Olivo Paulo, Brandon Arellano, Isaac Trujillo)

Quito, 07 de octubre de 2026

I INTRODUCCION

El presente informe documenta la implementación del patrón de integración "File Transfer" utilizando el framework Apache Camel. Este patrón permite la comunicación asíncrona entre sistemas desacoplados mediante el uso de un directorio de archivos compartido. Durante la práctica, se configuró una ruta de integración automatizada basada en Java que monitorea una carpeta de entrada, procesa documentos en formato CSV, transforma su contenido a texto en mayúsculas y los transfiere secuencialmente a directorios de salida y archivado, demostrando la capacidad de Camel para orquestar flujos de datos con mínima codificación.

II OBJETIVOS

El estudiante aprenderá a implementar un flujo de integración simple que lea un archivo CSV desde una carpeta de entrada, transforme su contenido y lo mueva a una carpeta de salida, aplicando el patrón clásico de integración por transferencia de archivos, usando Apache Camel.

III JUSTIFICACION

En la ingeniería de software, la integración de aplicaciones empresariales (EAI) es un desafío crítico, especialmente al conectar sistemas modernos con arquitecturas *legacy* que carecen de APIs o soporte para colas de mensajería. La realización de este laboratorio se justifica por la necesidad de dominar soluciones estándar de la industria para el procesamiento asíncrono. Implementar este flujo con Apache Camel y Maven proporciona competencias prácticas para diseñar procesos por lotes (*batch*), sincronizaciones de inventario y exportaciones de datos de forma escalable, segura y tolerante a fallos de red temporales.

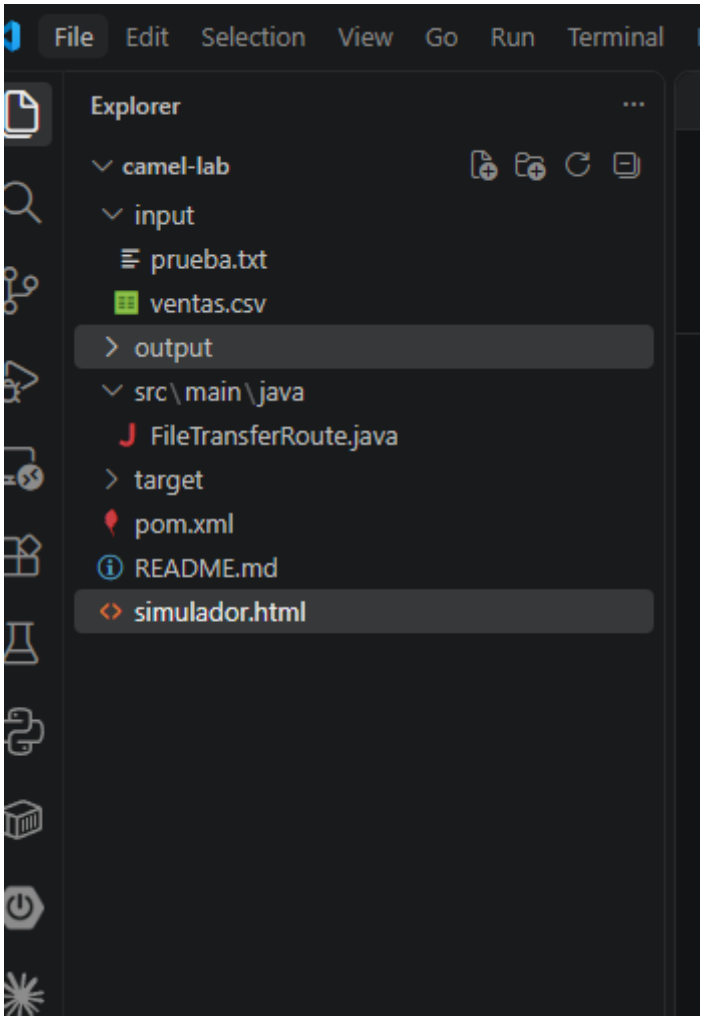
IV DESARROLLO

Después de instalar Apache Camel mediante Maven empezamos con el paso 1:

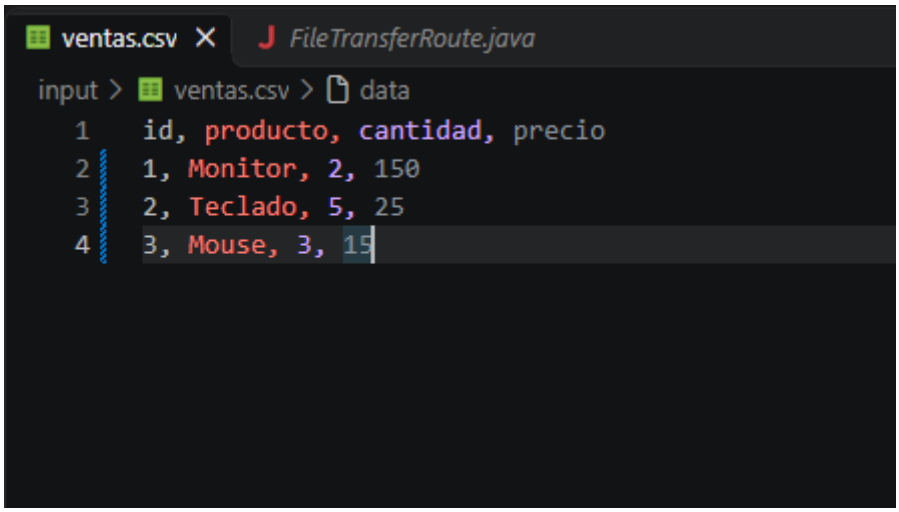
Creación de las estructuras de las carpetas de flujo

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

ISWZ 3401– INTELIGENCIA ARTIFICIAL I - [5448] - 202620



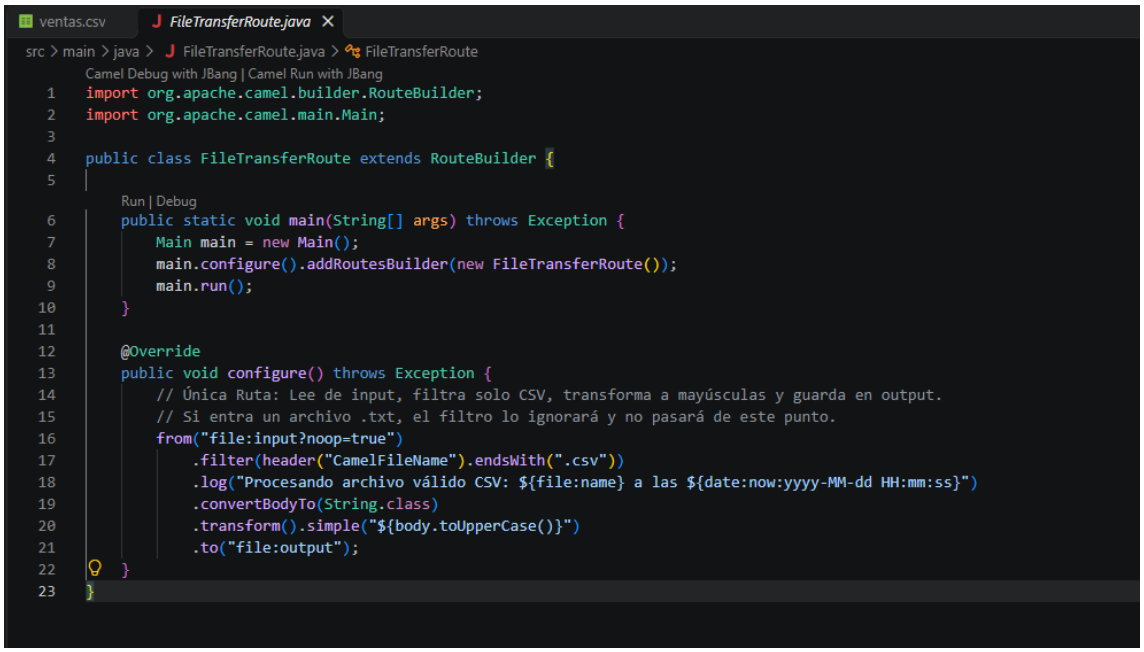
Creación de un archivo CSV de ejemplo



FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

ISWZ 3401– INTELIGENCIA ARTIFICIAL I - [5448] - 202620

Creación de un flujo básico en Apache Camel



```java
import org.apache.camel.builder.RouteBuilder;
import org.apache.camel.main.Main;

public class FileTransferRoute extends RouteBuilder {

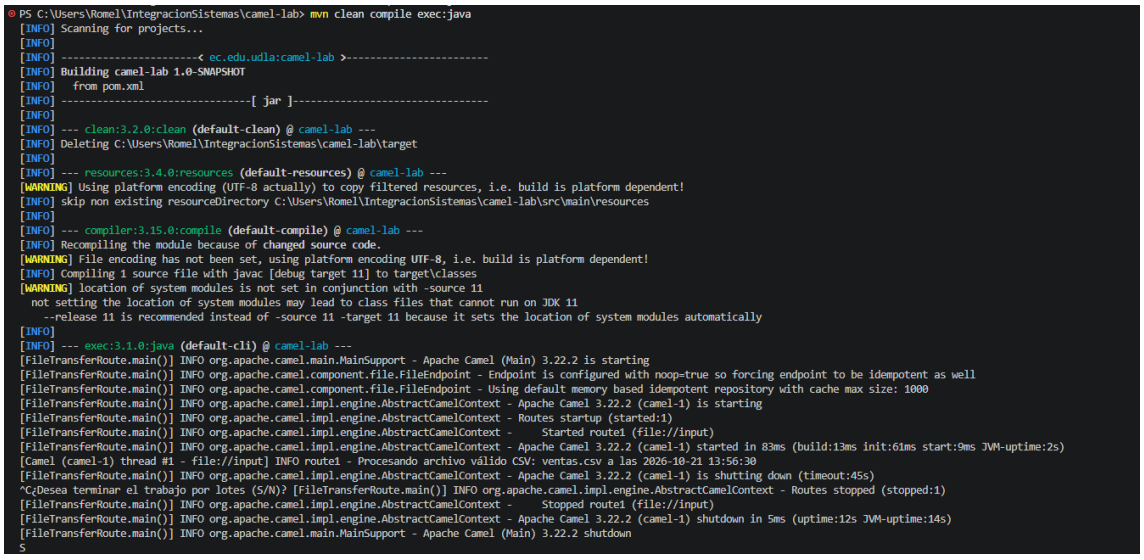
    public static void main(String[] args) throws Exception {
        Main main = new Main();
        main.configure().addRoutesBuilder(new FileTransferRoute());
        main.run();
    }

    @Override
    public void configure() throws Exception {
        // Única Ruta: Lee de input, filtra solo CSV, transforma a mayúsculas y guarda en output.
        // Si entra un archivo .txt, el filtro lo ignorará y no pasará de este punto.
        from("file:input?noop=true")
            .filter(header("CamelFileName").endsWith(".csv"))
            .log("Procesando archivo válido CSV: ${file:name} a las ${date:now:yyyy-MM-dd HH:mm:ss}")
            .convertBodyTo(String.class)
            .transform().simple("${body.toUpperCase()}")
            .to("file:output");
    }
}
```

Dentro de esta parte es donde se convierte de minúsculas a mayúsculas y además se mete el filtro para que solo sean validos los archivos csv

V CAPTURAS DE PANTALLA TERMINAL MOSTRANDO LOGS

Ejecución del flujo mediante consola y apache camel



```
PS C:\Users\Romel\IntegracionSistemas\camel-lab> mvn clean compile exec:java
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< ec.edu.udla:camel-lab >-----------------------
[INFO] Building camel-lab 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ camel-lab ---
[INFO] Deleting C:\Users\Romel\IntegracionSistemas\camel-lab\target
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ camel-lab ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\Users\Romel\IntegracionSistemas\camel-lab\src\main\resources
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ camel-lab ---
[INFO] Recompiling the module because of changed source code.
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file with javac [debug target 11] to target\classes
[WARNING] location of system modules is not set in conjunction with -source 11
  not setting the location of system modules may lead to class files that cannot run on JDK 11
    --release 11 is recommended instead of -source 11 -target 11 because it sets the location of system modules automatically
[INFO]
[INFO] --- exec:3.1.0:java (default-cli) @ camel-lab ---
[FileTransferRoute.main()] INFO org.apache.camel.main.MainSupport - Apache Camel (Main) 3.22.2 is starting
[FileTransferRoute.main()] INFO org.apache.camel.component.file.FileEndpoint - Endpoint is configured with noop=true so forcing endpoint to be idempotent as well
[FileTransferRoute.main()] INFO org.apache.camel.component.file.FileEndpoint - Using default memory based idempotent repository with cache max size: 1000
[FileTransferRoute.main()] INFO org.apache.camel.impl.engine.AbstractCamelContext - Apache Camel 3.22.2 (camel-1) is starting
[FileTransferRoute.main()] INFO org.apache.camel.impl.engine.AbstractCamelContext - Routes startup (started:1)
[FileTransferRoute.main()] INFO org.apache.camel.impl.engine.AbstractCamelContext -     Started route1 (file://input)
[FileTransferRoute.main()] INFO org.apache.camel.impl.engine.AbstractCamelContext - Apache Camel 3.22.2 (camel-1) started in 83ms (build:13ms init:61ms start:9ms JVM-uptime:2s)
[Camel (camel-1) thread #1 - file://input] INFO route1 - Procesando archivo válido CSV: ventas.csv a las 2026-10-21 13:56:30
[FileTransferRoute.main()] INFO org.apache.camel.impl.engine.AbstractCamelContext - Apache Camel 3.22.2 (camel-1) is shutting down (timeout:45s)
^C¿Desea terminar el trabajo por lotes (S/N)? [FileTransferRoute.main()] INFO org.apache.camel.impl.engine.AbstractCamelContext - Routes stopped (stopped:1)
[FileTransferRoute.main()] INFO org.apache.camel.impl.engine.AbstractCamelContext -     Stopped route1 (file://input)
[FileTransferRoute.main()] INFO org.apache.camel.impl.engine.AbstractCamelContext - Apache Camel 3.22.2 (camel-1) shutdown in 5ms (uptime:12s JVM-uptime:14s)
[FileTransferRoute.main()] INFO org.apache.camel.main.MainSupport - Apache Camel (Main) 3.22.2 shutdown
S
```

Aquí es cuando la consola ya nos muestra que el archivo de la carpeta input de tipo csv ya ha sido procesado correctamente y fue enviado a la carpeta output pero con el contenido en mayúsculas, dándonos información sobre a que hora fue procesado.

Los procesos específicos funcionan de esta manera:

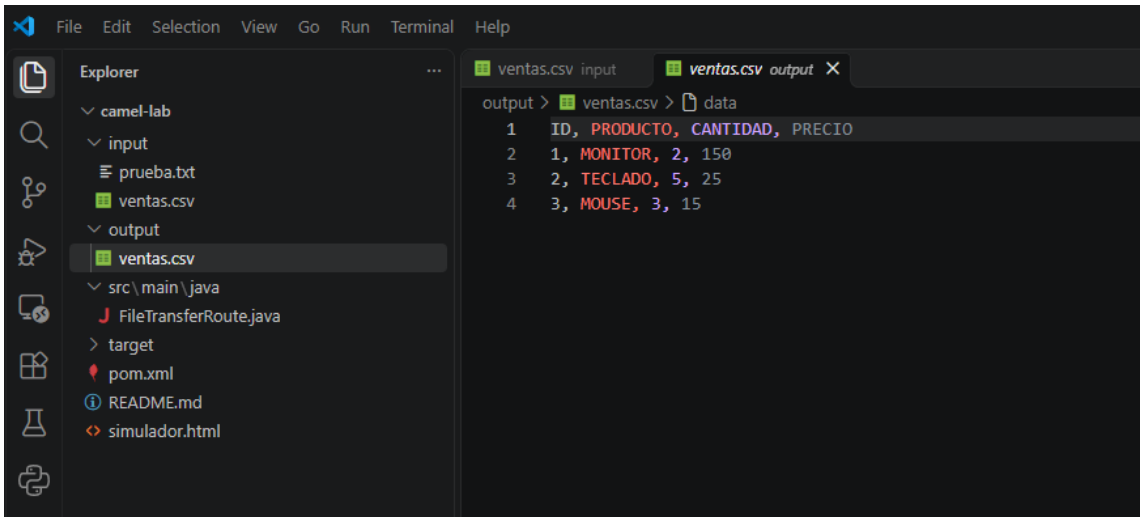
1. **Filtrado:** Utiliza .filter(header("CamelFileName").endsWith(".csv")) para procesar únicamente archivos con extensión CSV. Cualquier otro archivo (como .txt) es ignorado automáticamente.
2. **Transformación:** Convierte el cuerpo del archivo procesado a texto y transforma todo su contenido a mayúsculas utilizando el lenguaje simple de Camel (${body.toUpperCase()}).

VI CARPETA OUTPUT CON EL ARCHIVO TRANSFORMADO

Primer archivo base en input con el contenido en minusculas:



Archivo procesado y guardado en la carpeta output con el contenido en minúsculas:



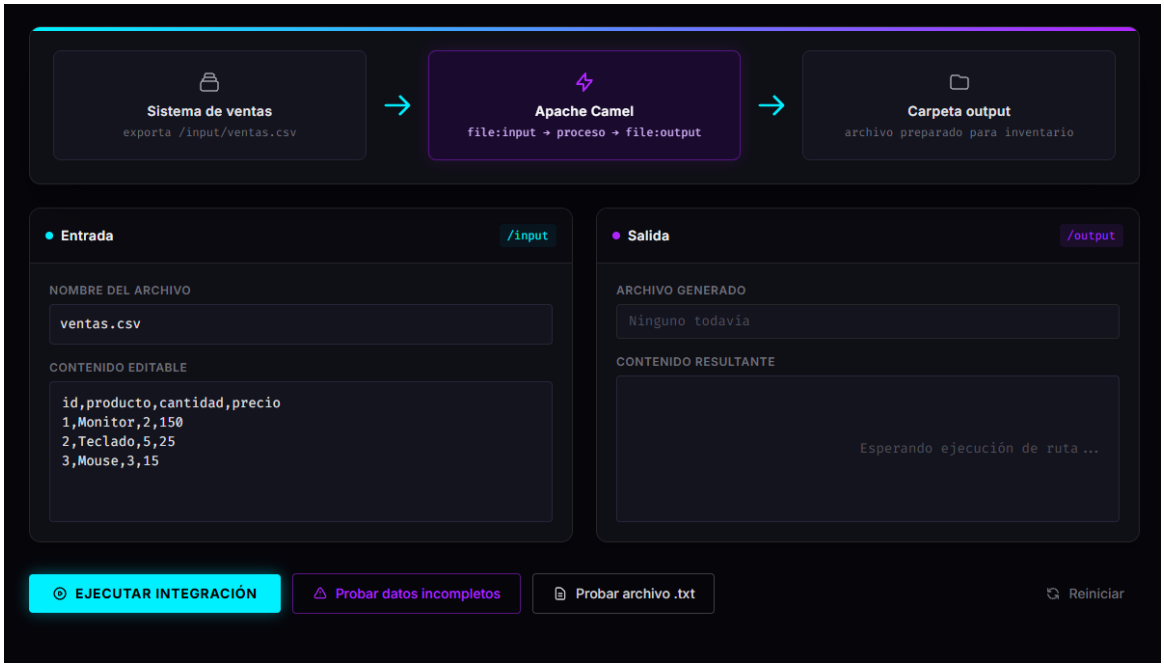
VII SIMULADOR DE FLUJO HTML

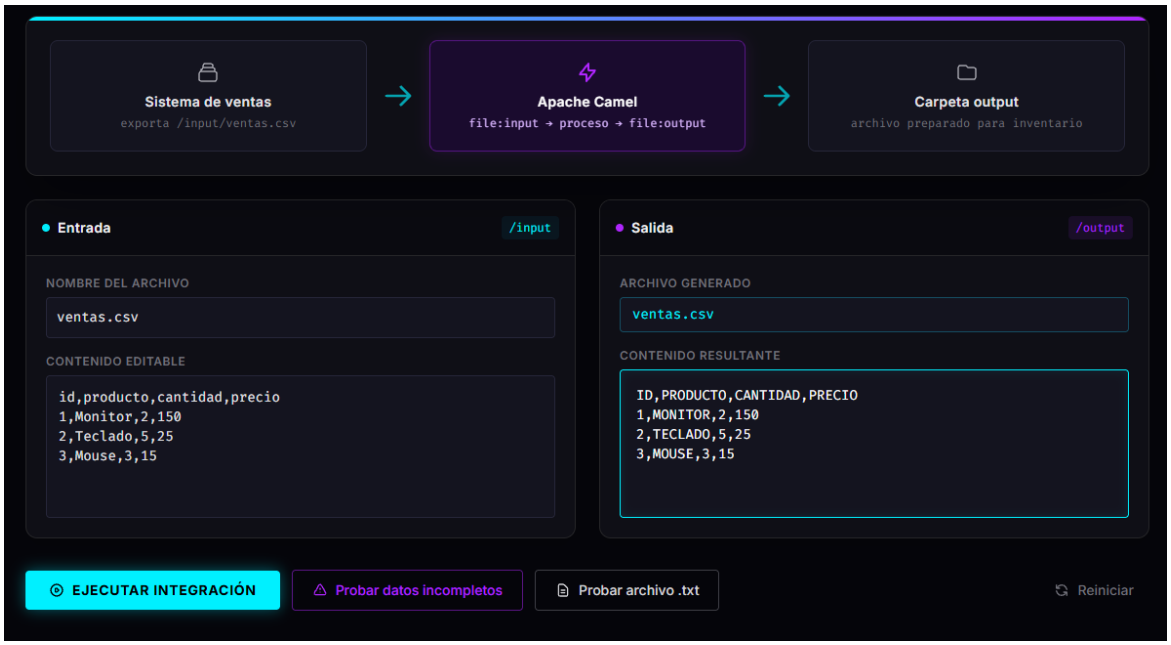
Adicionalmente hemos creado un simulador de flujo en html para ver como funciona nuestro programa, algo muy relacionado con el contenido presentado la clase anterior por

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

ISWZ 3401– INTELIGENCIA ARTIFICIAL I - [5448] - 202620

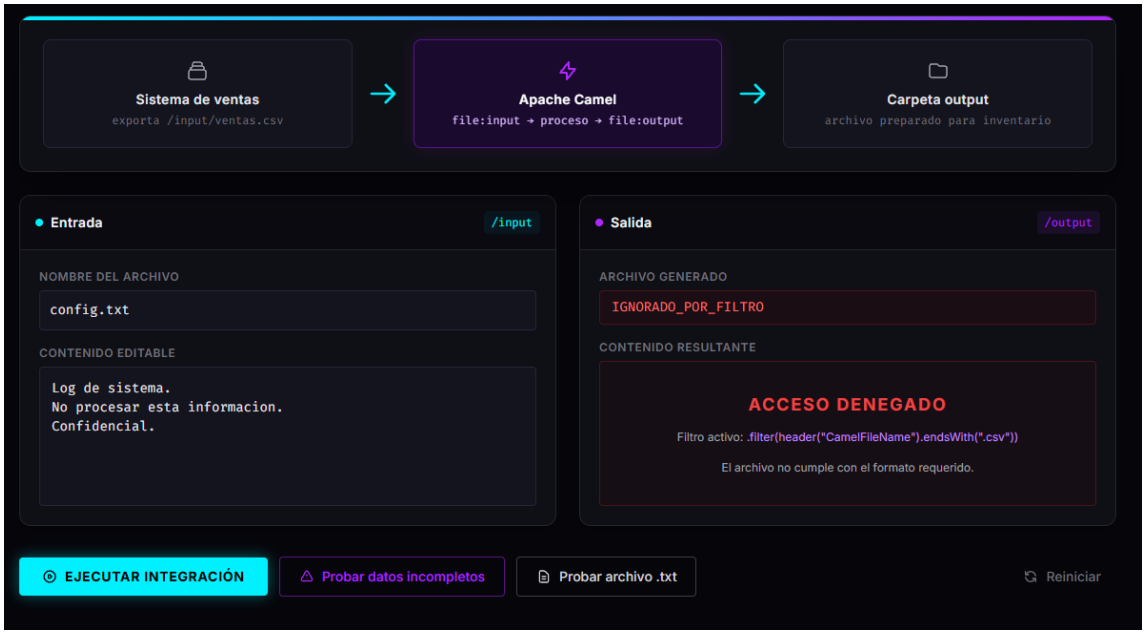
el docente





FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

ISWZ 3401– INTELIGENCIA ARTIFICIAL I - [5448] - 202620



VIII PREGUNTAS DE REFLEXION

¿Qué representa el patrón "File Transfer"?

Representa un método de integración de sistemas asíncrono donde una aplicación exporta información a un archivo físico y lo deposita en un directorio, para que otra aplicación lo lea y actualice sus registros. Actúa como un puente que desacopla los sistemas, permitiendo el intercambio de datos sin que las aplicaciones se comuniquen directamente o conozcan la arquitectura de la otra.

¿Qué limitaciones tiene este enfoque?

La principal limitación es la latencia, ya que los datos no se sincronizan en tiempo real sino mediante lotes programados. Además, carece de transaccionalidad y el manejo de errores es complejo.

¿En qué escenarios reales sería útil?

Es altamente útil para integrar sistemas antiguos que no soportan tecnologías modernas como APIs, Web Services o colas de mensajería, y cuya única vía de comunicación es la exportación de archivos. También es el estándar ideal para procesos por lotes, como actualizaciones masivas de inventario al final de la jornada, cierres contables nocturnos o conciliaciones bancarias donde no se requiere inmediatez.

IX CONCLUSIONES Y RECOMENDACIONES

Conclusiones:

- Se comprobó que el patrón File Transfer elimina la necesidad de conexiones de red directas y simultáneas entre aplicaciones, utilizando el sistema de archivos del sistema operativo como un intermediario neutro y confiable.

- La transición de una configuración manual de librerías a un entorno estructurado con Maven demostró ser crítica para evitar conflictos y asegurar la compatibilidad exacta entre el núcleo de Camel y las fachadas de registro (SLF4J).
- El uso del lenguaje de dominio específico de Apache Camel permitió definir reglas complejas de filtrado, transformación de tipos y ciclo de vida de los archivos (usando parámetros como delete=true) en apenas un par de líneas de código legibles.

Recomendaciones:

- Se recomienda iniciar futuros proyectos de integración directamente con gestores como Maven o Gradle, garantizando así la reproducibilidad del entorno en diferentes equipos y evitando errores de clases no encontradas.
- Para llevar el proyecto a un nivel de producción, se sugiere configurar un *Dead Letter Channel* en Camel. Esto permitirá capturar archivos CSV mal formateados y moverlos a una carpeta de "errores" sin detener el flujo de los demás archivos válidos.
- Una vez dominado el componente file, se aconseja escalar la práctica reemplazando las carpetas locales por servidores remotos utilizando los componentes ftp o sftp de Camel, acercando el modelo a un escenario empresarial distribuido real.

X CODIGO FUENTE EN GITHUB

https://github.com/Romelg18/camel-lab.git